■ Enterprise Software Development Blueprint: TotalPitch Management System

Project Name: TotalPitch (Football Team Management System)

Architect: Principal Software Architect @ Google

Status: Draft / For Review

Version: 1.0.0

■ Project Overview

TotalPitch is an enterprise-grade, cloud-native platform designed to digitize the end-to-end operations of professional football clubs. Unlike generic sports management tools, TotalPitch integrates tactical planning, player health monitoring, scouting intelligence, and administrative logistics into a single, high-performance ecosystem.

The system is built with a "Performance-First" philosophy, ensuring that coaches and medical staff have sub-second access to critical player data, while providing executive leadership with financial and strategic insights through advanced analytics.

■ Business Objective

1. **Operational Excellence:** Automate scheduling and logistics to reduce administrative overhead by 40%.

2. **Data-Driven Decisions:** Centralize player performance metrics to optimize training loads and reduce injury rates.

3. **Talent Management:** Streamline the scouting-to-signing pipeline with a unified recruitment module.

4. **Security & Compliance:** Ensure 100% compliance with GDPR and medical data privacy standards (HIPAA-equivalent).

■ User Roles

* **System Administrator:** Manages club settings, user permissions, and integrations.

* **Head Coach / Tactical Staff:** Manages training sessions, match tactics, and squad selection.

* **Medical Staff / Physio:** Tracks player injuries, rehabilitation progress, and fitness tests.

* **Scout:** Submits player reports and monitors external talent.

* **Player:** Accesses personal schedules, training videos, and wellness self-assessments.

* **Executive/Owner:** High-level dashboards for financial health and team ROI.

■ Functional Requirements

1. Squad & Player Management

* Complete digital profiles (Bio, Contract details, Career stats).

* Document management (Passports, Visas, Contracts).

* Dynamic depth charts for position-based squad planning.

2. Training & Match Logistics

* Calendar integration for training drills, team meetings, and travel.

* Matchday management: Lineups, substitutions, and real-time event logging.

* Integration with GPS tracking providers (e.g., Catapult, StatSports).

3. Medical & Performance Hub

* Injury logging and "Return to Play" (RTP) workflow tracking.

* Daily wellness surveys (Sleep quality, muscle soreness, stress levels).

* Integration with Lab results and Body Composition data.

4. Scouting & Recruitment

* Global player database with custom rating scales.

* Video analysis linking (Integration with Wyscout/Hudl).

* Transfer shortlist management.

■ Non-Functional Requirements

* **Scalability:** Horizontal scaling via Kubernetes to handle spikes during matchdays.

* **Availability:** 99.99% Uptime SLA using multi-region deployment.

* **Latency:** Global CDN for media assets and <200ms API response time.

* **Security:** AES-256 encryption for data at rest; TLS 1.3 for data in transit; OIDC/OAuth2 for Authentication.

* **Observability:** Full-stack tracing using OpenTelemetry and Google Cloud Operations Suite (Stackdriver).

■ Suggested Technology Stack

| Layer | Technology |

| :--- | :--- |

| **Frontend** | React 18, Next.js (App Router), Tailwind CSS, TanStack Query |

| **Backend** | Go (Golang) using Gin or Buffalo framework (for high concurrency) |

| **Database (Primary)** | Google Cloud SQL (PostgreSQL) |

| **Cache/Real-time** | Redis (Memorystore) & WebSockets |

| **Auth** | Google Identity Platform / Firebase Auth (RBAC enabled) |

| **File Storage** | Google Cloud Storage (Bucket-based media management) |

| **DevOps** | Terraform (IaC), GitHub Actions, Google Kubernetes Engine (GKE) |

■ Suggested Folder Structure

```text
totalpitch-monorepo/
■■■ apps/
■ ■■■ web-dashboard/ # Next.js admin/staff portal
■ ■■■ mobile-app/ # React Native player app
■■■ services/
■ ■■■ api-gateway/ # Central entry point (Nginx/Envoy)
■ ■■■ core-service/ # Go: Player, Team, & User management
■ ■■■ activity-service/ # Go: Training & Match logic
■ ■■■ medical-service/ # Go: Sensitive health data (encrypted)
■■■ packages/
■ ■■■ ui-kit/ # Shared Tailwind components
■ ■■■ database-schema/ # Prisma/SQL migration files
■ ■■■ internal-lib/ # Shared Go utilities & logging
■■■ infrastructure/
■ ■■■ terraform/ # GCP Resource definitions
■■■ README.md
```

■ REST API Endpoints (Sample)

Method	Endpoint	Purpose
`GET`	`/api/v1/players`	Retrieve all players with filters (age, position).
`POST`	`/api/v1/players`	Register a new player to the squad.
`GET`	`/api/v1/players/{id}/medical`	Retrieve medical history (Role-restricted).
`POST`	`/api/v1/matches`	Schedule a new match.
`PATCH`	`/api/v1/matches/{id}/lineup`	Update starting XI and bench.
`GET`	`/api/v1/scouting/reports`	Fetch scouting reports for a specific target.
`POST`	`/api/v1/wellness/daily`	Submit daily player wellness data.

■ Database Design (Relational)

Table: `users`

* `id` (UUID, PK), `email` (String, Unique), `password_hash`, `role_id` (FK), `created_at`.

Table: `players`

* `id` (UUID, PK), `user_id` (FK), `first_name`, `last_name`, `date_of_birth`, `nationality`, `position`, `contract_expiry`.

Table: `events` (Training/Matches)

* `id` (UUID, PK), `type` (Enum: 'match', 'training'), `start_time`, `end_time`, `location`, `opponent_id` (Nullable).

Table: `medical_records`

* `id` (UUID, PK), `player_id` (FK), `injury_type`, `severity`, `status` (Enum), `estimated_return`, `notes` (Encrypted).

■ Deployment Architecture

1. **Traffic Entry:** Google Cloud Load Balancer (GCLB) provides a single Global IP.

2. **Security:** Cloud Armor protects against DDoS and SQL injection.

3. **Compute:** GKE (Google Kubernetes Engine) runs containerized microservices in Autopilot mode for cost efficiency.

4. **Database:** Cloud SQL (PostgreSQL) with High Availability (HA) enabled across two zones.

5. **Caching:** Cloud Memorystore (Redis) for session management and leaderboard/stat caching.

6. **CI/CD:** GitHub Actions triggers Google Cloud Build to push images to Artifact Registry and update GKE manifests.

■ Development Roadmap

Week 1: Foundation & Infrastructure

* Setup GCP Project and Terraform scripts.

* Configure Auth (Google Identity Platform).

* Initialize Monorepo and CI/CD pipelines.

Week 2: Core Squad Management

* Develop Player and User CRUD APIs.

* Build the "Squad Overview" dashboard UI.

* Implement Role-Based Access Control (RBAC).

Week 3: Calendar & Logistics

* Develop Scheduling Engine (Training/Matches).

* Implement timezone-aware calendar views.

* Push Notification service for player reminders.

Week 4: Medical & Performance Tracking

* Build Medical logging system with field-level encryption.

* Implement Wellness Survey module.

* Integration with fitness CSV/API data.

Week 5: Scouting & Analytics

* Develop Scouting Report builder.

* Build the Executive Dashboard (Aggregated stats).
* Global search functionality (Elasticsearch or Postgres Full-text).

Week 6: QA, Security Audit & Launch
* Load testing (k6) and Penetration testing.
* Final UAT (User Acceptance Testing) with staff.
* Production rollout and monitoring setup.

Prepared by:

Principal Software Architect

Google Cloud - Professional Services Organization